

Software Engineering II

Container

Mathis Neunzig
ABAP Developer at SAP
mathis.neunzig@gmail.com



Disclaimer

- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesung sind kein Ersatz für Literatur und können nicht als Quellen für wissenschaftliche Arbeiten genutzt werden.
- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesungen dürfen nur von den Studierenden des Kurses WI 2024 SE A zur Nachbereitung der Vorlesung genutzt werden. Eine kommerzielle Nutzung und Weitergabe ist verboten.
- Alle Präsentation sowie alle weiteren Materialien werden vor Beginn der jeweiligen Vorlesung hochgeladen.
- Zur besseren Lesbarkeit wird auf die gleichzeitige Nutzung von weiblichen & männlichen Begriffen verzichtet. Es wird das generische Maskulinum verwendet. Dabei sind weibliche und sonstige Formen des Wortes gleichwertig mitgemeint. (Beispiel: statt "Kund:innen" → „Kunden“)

Problem

- „It works on my machine“
- „How do I install npm?“
- „Ah ***, I have the wrong node version...“
- „How do I set this up???“

Docker

- Open-Source-Plattform zur Containerisierung
- Anwendungen in isolierte Container packen & ausführen
- Software läuft unabhängig von der Infrastruktur
- Container beinhaltet alle Bibliotheken und den auszuführenden Code

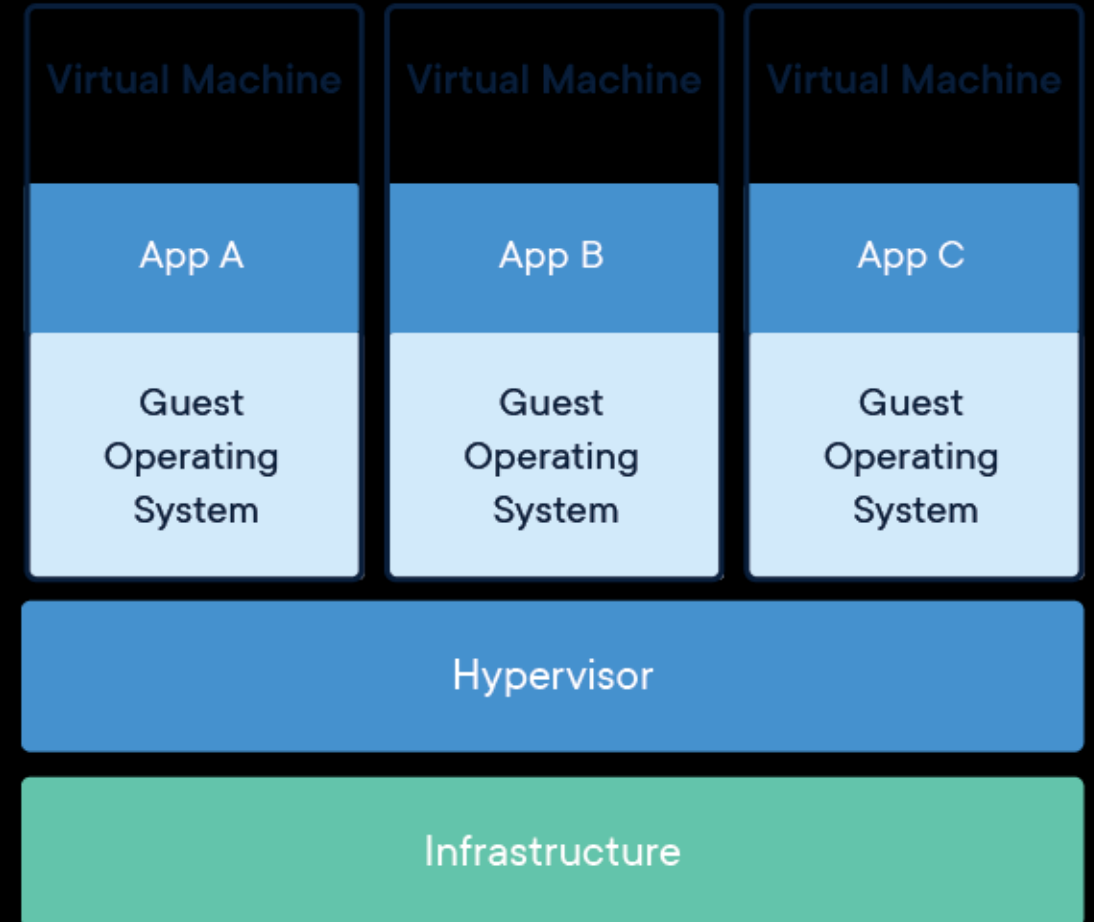
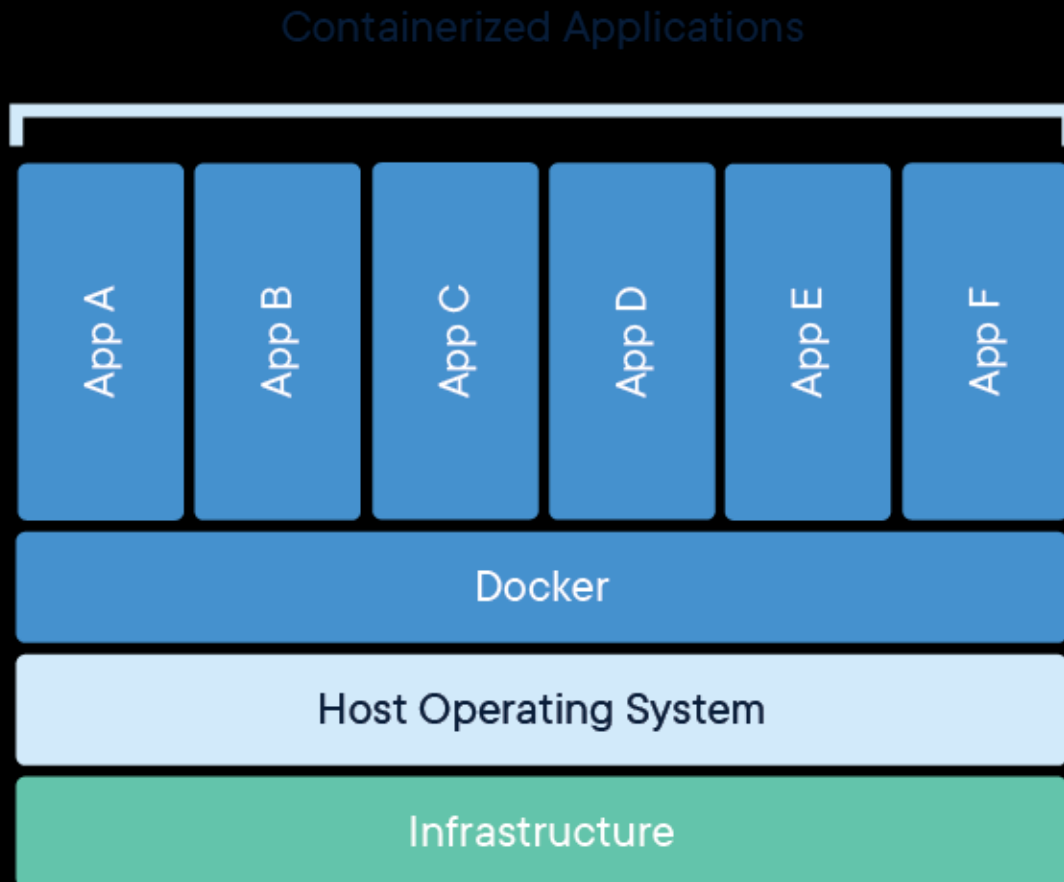


Container vs. Virtuelle Maschinen

- Container sind leichtgewichtiger und effizienter als traditionelle virtuelle Maschinen
- Container nutzen den Kernel des Host-Betriebssystems mit
- Container teilen sich viele Ressourcen
- Virtuelle Maschinen erfordern ein komplettes Betriebssystem pro Instanz

→ Es ist recht einfach, viele Container auf einem Rechner parallel laufen zu lassen

Container vs. Virtuelle Maschinen



Ein paar Begriffe

- Ein Docker Image ist eine schreibgeschützte Vorlage, die den Zustand eines Containers definiert
- Der Docker Daemon führt Container aus und verwaltet Images
- Das Docker CLI (Command Line Interface) stellt Befehle bereit, um mit dem Daemon zu kommunizieren
- Mit Docker Desktop kann man über eine einfache Anwendung den Daemon bedienen
- In der Docker Registry werden die Images gespeichert und geteilt

Ein paar Begriffe

- Ein Docker Image ist eine schreibgeschützte Vorlage, die den Zustand eines Containers definiert
- Der Docker Daemon führt Container aus und verwaltet Images
- Das Docker CLI (Command Line Interface) stellt Befehle bereit, um mit dem Daemon zu kommunizieren
- Mit Docker Desktop kann man über eine einfache Anwendung den Daemon bedienen
- In der Docker Registry werden die Images gespeichert und geteilt

Dockerfile

- Eine Dockerfile beschreib ein Docker-Image

```
FROM node:14
```

```
WORKDIR /app
```

```
COPY src ./
```

```
RUN npm install
```

```
CMD ["npm", "run", "start"]
```

Dockerfile

- FROM
 - FROM ubuntu:20.04
 - Legt das Basis-Image fest, auf dem alle folgenden Befehle aufbauen
- LABEL
 - LABEL maintainer="mathis.neunzig@sap.com"
 - Fügt Metadaten zum Image hinzu, wie etwa Autor oder Versionsinformationen
- RUN
 - RUN npm install
 - Führt einen Befehl während des Build-Prozesses aus

Dockerfile

- CMD
 - CMD [„npm“, „run“, „dev“]
 - Bestimmt den Startbefehl der Anwendung
- COPY
 - COPY . /app
 - Kopiert die Daten vom aktuellen Kontext (.) auf das Image (/app)
- ADD
 - ADD test.zip /app
 - Wie COPY, erlaubt aber das Arbeiten mit Archiven

Dockerfile

- ENV
 - ENV API_KEY=„123456789“
 - Setzt Umgebungsvariablen, die in der App gebraucht werden
- EXPOSE
 - EXPOSE 8080
 - Dokumentiert, welcher Port zur Laufzeit genutzt wird (nur Doku!)
- WORKDIR
 - WORKDIR /app
 - Legt das Arbeitsverzeichnis fest, in dem die nachfolgenden Befehle ausgeführt werden

Dockerfile

- USER
 - USER appuser
 - Gibt an, welcher User die Befehle im Container ausführt
- #
 - # Ich bin ein Kommentar
 - Markiert einen Kommentar

Container

- Ein Image alleine macht nichts
- Ausführung eines Images als Container
- Container hat einige Vorteile
 - Stateless
 - Portable
 - Können auf vielen Cloud-Diensten laufen (AWS, Azure, GCP, ...)
- Um ein Image als Container auszuführen, muss das Image gebaut werden → Docker Build!

Docker Build

- Command
 - `docker build <location> <options>`
 - `docker build . -t DrBackmischung/myapp`
- Das Image wird in verschiedenen Schichten gebaut
- Beim erneuten Bauen werden nur solche Schichten neu gebaut, die auch geändert worden sind
- Aber der Container wird noch nicht ausgeführt! → Docker Run

Docker Run

- Command
 - `docker run <tag> <options>`
 - `docker run DrBackmischung/myapp -p 3000:3000`
- Das Image wird als Container ausgeführt
- 2 Wege zum Beenden
 - `docker container stop <name>`
 - `docker container kill <name>`

Docker Tag

- Command
 - `docker tag <tag> <second tag>`
 - Empfohlen: `<username>/<imagename>:<version>`
 - `docker tag DrBackmischung/myapp DrBackmischung/myapp:v1`
- Dem Image wird ein Tag gegeben
- Versionsmanagement

Docker Push

- Login
 - `docker login -u <username> -p <password> <repository>`
`<repository> → docker.io`
- Push
 - `docker push <image> <options>`
 - `docker push mynpmapp`
 - Bei einem Push muss der Username im Tag enthalten sein!
- Dies sorgt dafür, dass der Container in eine Registry hochgeladen wird
- Danach kann der Container in der Cloud genutzt werden
- Gegenstück: `docker pull <tag>`

Wie kann ich eine JS-App „Containerisieren“?

1. Account bei Docker erstellen
2. Dockerfile anlegen
3. `docker build . -t DrBackmischung/myapp`
4. `docker run DrBackmischung/myapp -p 3000:3000`

Wie kann ich eine JS-App „Containerisieren“?

1. Account bei Docker erstellen
2. Dockerfile anlegen
3. `docker build . -t DrBackmischung/myapp`
4. `docker tag DrBackmischung/myapp DrBackmischung/myapp:v1`
5. `docker login -u DrBackmischung -p Passwort docker.io`
6. `docker push DrBackmischung/myapp:v1`

.dockerignore

- Pendant zu .gitignore
- Aufgelistete Dateien werden nicht im Image enthalten sein
- Geeignet für
 - node_modules
 - Notizen
 - Konfigurationsdateien der IDE

Aber ich habe unendlich viele Container?

- Docker Compose
- Managen von Multi-Container-Applications
- Format: YAML
- Build: `docker-compose build`
- Start: `docker-compose up`
- Stop: `docker-compose down`

Übung: Docker

Ziel: Docker Container erstellen & etwas Übung mit GitHub

- Erstellt eine Fork vom Docker-Repository
- Erstellt einen neuen Ordner mit eurem/n Namen (Vorname(n))
- Entwickelt irgendwas in irgendeiner Programmiersprache und lasst es
- Erstellt eine Dockerfile, die euer Programm startet
- Öffnet eine Pull-Request
- Füllt die Beschreibung der Pull-Request aus!

Danke!

Mathis Neunzig
mathis.neunzig@gmail.com

